Een bruikbare manier om OOP te begrijpen is niet te beginnen bij "classes, objecten en method calls", maar bij één vraag:

> Wie is waarvoor verantwoordelijk?

Een object heeft een stukje verantwoordelijkheid. Om die verantwoordelijkheid uit te voeren heeft het object:

- eigen rauwe data;
- eventueel andere objecten waar het verantwoordelijkheden aan kan delegeren;
- methoden waarmee anderen vragen aan het object kunnen stellen of taken kunnen geven.

Daarmee bouw je uiteindelijk een hiërarchie van verantwoordelijkheid.

1. Wat bezit een object?

Stel dat we een webshop hebben:

```
class Order {
    private Customer customer;
    private List<OrderLine> orderLines;
    private boolean paid;
}
```

Je kunt de variabelen van `Order` op drie manieren bekijken.

`boolean paid` is rauwe data. `Order` hoeft niemand iets te vragen om te weten of hij betaald is. Die informatie heeft hij zelf.

`Customer customer` is een ander object. Dat kun je zien als een object waaraan `Order` vragen kan stellen:

```
customer.getDeliveryAddress()
customer.isAdult()
```

`List<OrderLine> orderLines` is een tussenvorm. De `List` is zelf een object, maar vooral een container waarin `Order` zijn onderliggende objecten bewaart.

Je mentale plaatje is dus ongeveer:

```
Order
│
├── paid                    ← rauwe data
│
├── customer                ← object dat Order dingen kan vragen
│
└── orderLines              ← container
      ├── OrderLine
      ├── OrderLine
      └── OrderLine
```

Dat is belangrijker dan het Java-detail dat dit "fields" zijn.

---

2. Methoden zijn vragen beantwoorden of taken uitvoeren

Bijna iedere methode kun je in normale mensentaal vertalen naar één van deze twee dingen:

Vraag: "Vertel mij iets."

```
order.getTotalPrice()
customer.getDeliveryAddress()
product.isInStock()
```

Of:

Taak: "Doe iets."

```
order.pay()
shoppingCart.addProduct(product)
printer.print(document)
```

Dat maakt code vaak veel gemakkelijker te begrijpen.

Als je dit ziet:

```
order.getTotalPrice();
```

hoef je niet eerst te denken:

> "Er wordt een methode aangeroepen op een instantie van de klasse Order."

Je kunt eerst denken:

> "Ik vraag de bestelling wat zijn totale prijs is."

De technische termen komen daarna wel.

Voorbeeld 1: een bestelling in een webshop

Neem deze structuur:

```
class Order {
    private Customer customer;
    private List<OrderLine> orderLines;
    private boolean paid;
}
```

Een `Order` is verantwoordelijk voor de bestelling als geheel.

Een `OrderLine` is verantwoordelijk voor één regel:

```
class OrderLine {
    private Product product;
    private int quantity;
}
```

En `Product` is verantwoordelijk voor productinformatie:

```
class Product {
    private String name;
    private double price;
}
```

De hiërarchie wordt dan:

```
Order
│
├── Customer
│
└── meerdere OrderLines
       │
       └── Product
```

Iedereen weet alleen wat hij hoeft te weten.

Vraag: "Wat kost deze bestelling?"

Die vraag stel je aan `Order`:

```
double total = order.getTotalPrice();
```

In gewone taal:

> Order, wat kost jij in totaal?

Maar `Order` kent niet zelf de prijs van ieder product. Dus `Order` vraagt het aan zijn ondergeschikten:

```
public double getTotalPrice() {
    double total = 0;

    for (OrderLine line : orderLines) {
        total += line.getTotalPrice();
    }

    return total;
}
```

`Order` zegt dus eigenlijk:

> OrderLine 1, wat kost jij?
> OrderLine 2, wat kost jij?
> OrderLine 3, wat kost jij?

Een `OrderLine` weet dat ook niet volledig uit zichzelf:

```
public double getTotalPrice() {
    return product.getPrice() * quantity;
}
```

Die zegt:

> Product, wat kost jij?

En `Product` kan die vraag zelf beantwoorden:

```
public double getPrice() {
    return price;
}
```

Want uiteindelijk komen we bij rauwe data:

```
"Wat kost de bestelling?"
        │
        ▼
      Order
        │
        │ vraagt:
        ▼
    OrderLine
        │
        │ vraagt:
        ▼
      Product
        │
        ▼
    price = 12.50
```

Dat is OOP als verantwoordelijkheidshiërarchie.

De bovenste objecten hoeven niet alle details te kennen. Ze weten vooral **aan wie ze iets moeten vragen**.

Voorbeeld 2: een auto

Een heel tastbaar voorbeeld:

```
class Car {
    private Engine engine;
    private FuelTank fuelTank;
    private boolean locked;
}
```

De auto heeft:

```
Car
│
├── locked       ← eigen rauwe data
├── Engine       ← ondergeschikt object
└── FuelTank     ← ondergeschikt object
```

Stel dat iemand vraagt:

```
car.canDrive();
```

In normale taal:

> Auto, kun jij rijden?

Om die vraag te beantwoorden heeft de auto informatie van zijn ondergeschikten nodig.

```
public boolean canDrive() {
    return engine.isWorking()
        && fuelTank.hasFuel();
}
```

De auto vraagt dus:

> Motor, werk jij?

en:

> Brandstoftank, heb jij nog brandstof?

De `Engine` kan bijvoorbeeld zelf rauwe data bezitten:

```
class Engine {
    private boolean broken;

    public boolean isWorking() {
        return !broken;
    }
}
```

En de tank:

```
class FuelTank {
    private double liters;

    public boolean hasFuel() {
        return liters > 0;
    }
}
```

De verantwoordelijkheid is verdeeld:

```
Car
│
│ "Kan ik rijden?"
│
├── vraagt Engine:
│      "Werk jij?"
│
└── vraagt FuelTank:
       "Heb jij brandstof?"
```

`Car` hoeft niet te weten hoe een motor intern bepaalt of hij werkt.

Dat is juist het punt.

De auto zegt niet:

> "Laat mij even jouw interne variabelen bekijken zodat ik zelf kan uitrekenen of jij werkt."

Hij vraagt gewoon:

> "Motor, werk jij?"

Voorbeeld 3: restaurant

Dit voorbeeld werkt goed om het verschil tussen **vragen** en **taken uitdelen** duidelijk te maken.

Stel:

```
class Restaurant {
    private Kitchen kitchen;
    private CashRegister cashRegister;
    private List<Table> tables;
}
```

Een klant wil eten bestellen.

De restaurantklasse hoeft niet zelf het eten te koken.

Hij kan zeggen:

```
kitchen.prepare(order);
```

In mensentaal:

> Keuken, bereid deze bestelling.

Dat is taakdelegatie.

De `Kitchen` heeft misschien weer:

```
class Kitchen {
    private Oven oven;
    private List<Cook> cooks;
}
```

De keuken kan vervolgens verder delegeren:

```
oven.preheat(220);
cook.preparePizza();
```

Dus:

```
Restaurant
    │
    │ "Maak bestelling 42"
    ▼
Kitchen
    │
    ├── "Oven, verwarm naar 220 graden"
    │
    └── "Kok, maak de pizza"
```

Geen enkel object doet per se alles zelf.

Een object is verantwoordelijk voor een taak, maar **verantwoordelijk zijn betekent niet dat je alles eigenhandig moet uitvoeren**.

Dat is een belangrijk OOP-principe.

Een restaurantmanager is verantwoordelijk dat de maaltijd op tafel komt.

Dat betekent niet dat de manager zelf:

- de oven verwarmt;
- groenten snijdt;
- bakt;
- de rekening print.

De manager heeft mensen en onderdelen aan wie hij die taken kan delegeren.

Zo werken objecten ook.

Het kernmodel

Voor iedere class kun je eigenlijk drie vragen stellen.

1. Welke informatie is mijn verantwoordelijkheid?

Bijvoorbeeld `Product`:

```
class Product {
    private String name;
    private double price;
}
```

`Product` is verantwoordelijk voor:

> Wat is mijn naam?
> Wat kost ik?

Die antwoorden komen rechtstreeks uit zijn eigen data.

---

2. Welke vragen kan ik beantwoorden / welke taken kan ik uitvoeren?

Bijvoorbeeld:

```
product.getPrice();
product.isExpensive();
```

Dat zijn vragen.

```
order.pay();
car.start();
shoppingCart.addProduct(product);
```

Dat zijn taken.

Soms kan een object die volledig zelf uitvoeren.

Soms moet het daarvoor dingen aan ondergeschikte objecten vragen.

---

3. Aan wie kan ik iets vragen of delegeren?

Antwoord:

> Aan de objecten die ik als klassevariabelen bezit.

Bijvoorbeeld:

```
class Car {
    private Engine engine;
    private Navigation navigation;
}
```

De auto kan tegen `engine` zeggen:

> Start.

En tegen `navigation`:

> Bereken een route naar Amsterdam.

```
engine.start();
navigation.calculateRoute("Amsterdam");
```

De auto kan niet zomaar tegen een willekeurige `Chef`, `BankAccount` of `Spaceship` iets zeggen. Hij heeft daar geen relatie mee.

Zijn mogelijke medewerkers zie je dus letterlijk terug in zijn object-typed velden.

Primitive versus object

Dit onderscheid kun je heel praktisch uitleggen als:

```
private int age;
```

betekent:

> Ik bezit zelf het antwoord.

Terwijl:

```
private Customer customer;
```

betekent:

> Ik ken iemand aan wie ik dingen kan vragen.

Bijvoorbeeld:

```
class Order {
    private boolean paid;
    private Customer customer;
}
```

Voor:

```
isPaid()
```

kijkt `Order` gewoon naar zijn eigen rauwe data:

```
public boolean isPaid() {
    return paid;
}
```

Maar voor:

```
getDeliveryAddress()
```

kan hij delegeren:

```
public Address getDeliveryAddress() {
    return customer.getDeliveryAddress();
}
```

Dus:

```
primitive field
    ↓
"Ik weet het zelf."


object field
    ↓
"Ik weet aan wie ik het kan vragen."
```

Dat is een zeer bruikbaar mentaal model.

Containers zijn een kleine uitzondering

Neem:

```
private List<Student> students;
```

De `List` zelf is technisch gezien ook een object.

Je kunt hem vragen:

```
students.size();
students.add(student);
students.remove(student);
```

Maar conceptueel gebruik je hem meestal als:

> mijn verzameling ondergeschikte objecten.

Bijvoorbeeld:

```
Classroom
│
└── List<Student>
      ├── Student
      ├── Student
      └── Student
```

`Classroom` kan dan bijvoorbeeld vragen:

```
public int getNumberOfPresentStudents() {
    int total = 0;

    for (Student student : students) {
        if (student.isPresent()) {
            total++;
        }
    }

    return total;
}
```

In gewone taal:

> Leerling 1, ben je aanwezig?
> Leerling 2, ben je aanwezig?
> Leerling 3, ben je aanwezig?

De klas verzamelt vervolgens de antwoorden.

Een belangrijk verschil: vragen versus in andermans administratie neuzen

Hier wordt OOP interessant.

Stel dat een `BankAccount` geld bevat:

```
class BankAccount {
    private double balance;
}
```

Andere objecten zouden eigenlijk niet moeten zeggen:

> Geef mij jouw `balance`, dan ga ik zelf bepalen wat ermee moet gebeuren.

Liever geef je het account verantwoordelijkheden:

```
account.canWithdraw(100);
account.withdraw(100);
```

Dus niet:

```
if (account.getBalance() >= 100) {
    account.setBalance(account.getBalance() - 100);
}
```

Dat betekent eigenlijk:

> Rekening, geef mij al je interne informatie. Ik neem jouw verantwoordelijkheid wel over.

Maar:

```
account.withdraw(100);
```

betekent:

> Rekening, haal €100 van jezelf af.

De rekening kan vervolgens zelf regels toepassen:

```
public void withdraw(double amount) {
    if (amount <= 0) {
        throw new IllegalArgumentException();
    }

    if (amount > balance) {
        throw new IllegalStateException();
    }

    balance -= amount;
}
```

Dat is een van de belangrijkste gevolgen van deze manier van denken:

> Je vraagt een object bij voorkeur iets te **doen**, in plaats van zijn data op te vragen zodat je het daarna zelf gaat doen.

Een slecht en goed voorbeeld naast elkaar

Stel een `Game` heeft een speler:

```
class Game {
    private Player player;
}
```

De speler heeft health:

```
class Player {
    private int health;
}
```

Een vijand doet 20 damage.

Minder goede verantwoordelijkheid:

```
int health = player.getHealth();
player.setHealth(health - 20);
```

Vertaling:

> Speler, hoeveel health heb je?
> Oké, dan reken ik zelf uit hoeveel je straks hebt.
> Speler, verander je health nu naar deze waarde.

`Game` is hier bezig met iets wat eigenlijk de verantwoordelijkheid van `Player` is.

Beter:

```
player.takeDamage(20);
```

Vertaling:

> Speler, ontvang 20 damage.

En de speler bepaalt zelf wat dat betekent:

```
public void takeDamage(int damage) {
    health -= damage;

    if (health < 0) {
        health = 0;
    }
}
```

Misschien heeft de speler later armor:

```
class Player {
    private int health;
    private Armor armor;
}
```

Dan kan `Player` weer delegeren:

```
public void takeDamage(int damage) {
    int remainingDamage = armor.absorbDamage(damage);
    health -= remainingDamage;
}
```

De rest van het programma hoeft niet aangepast te worden.

Nog steeds:

```
player.takeDamage(20);
```

Maar intern ontstaat:

```
Game
 │
 │ "Player, ontvang 20 damage"
 ▼
Player
 │
 │ "Armor, hoeveel daarvan kun jij tegenhouden?"
 ▼
Armor
 │
 ▼
"Ik houd 8 tegen"
 │
 ▼
Player verliest 12 health
```

Hier zie je de verantwoordelijkheidshiërarchie heel duidelijk.

De hiërarchie is belangrijker dan de losse classes

Een programma kun je daarom zien als een organisatie.

Bijvoorbeeld:

```
Webshop
│
├── ShoppingCart
│    └── CartItems
│          └── Products
│
├── Customer
│    └── Address
│
└── PaymentService
     └── PaymentProvider
```

Wanneer bovenaan iets moet gebeuren, wordt de verantwoordelijkheid naar beneden verdeeld.

Bijvoorbeeld:

```
Webshop:
"Checkout deze winkelwagen."

        ↓

ShoppingCart:
"Hoeveel kost ik?"

        ↓

CartItem:
"Hoeveel kost ik?"

        ↓

Product:
"Mijn prijs is €20."
```

En daarna:

```
Webshop:
"PaymentService, betaal €60."

        ↓

PaymentService:
"PaymentProvider, voer deze betaling uit."
```

Dat is een veel bruikbaarder beeld van OOP dan:

> object A roept methode B aan op object C.

Technisch klopt dat, maar het verbergt waarom je het doet.

De samenvatting die ik hiervoor zou gebruiken

Ik zou het uiteindelijk reduceren tot deze drie regels:

> **Mijn primitive variabelen zijn dingen die ik zelf weet.**

> **Mijn objectvariabelen zijn andere objecten aan wie ik vragen kan stellen of taken kan geven.**

> **Mijn methoden zijn de vragen die ik kan beantwoorden en de taken die ik kan uitvoeren.**

En daaruit volgt:

> **Als ik iets niet zelf kan beantwoorden of uitvoeren, delegeer ik het naar een object waarvoor ik verantwoordelijk ben.**

Zo ontstaat vanzelf een boom:

```
Object
│
├── wat weet ik zelf?
│      └── primitives / eigen data
│
├── voor wie ben ik verantwoordelijk?
│      └── object fields / collections
│
└── wat kan men mij vragen?
       └── methods
             │
             ├── zelf beantwoorden
             │
             └── verder delegeren
```

Als leerlingen dit model eenmaal zien, worden termen als `object`, `field`, `method`, `encapsulation`, `composition` en zelfs zaken als de Law of Demeter veel minder abstract: het zijn grotendeels technische namen voor **wie weet wat, wie doet wat, en wie mag het aan wie vragen**.